

Lab 7: Database Connectivity — Game Catalog CRUD

ชื่อ-นามสกุล: นางสาวกิตติญาดา กองคำ รหัสนักศึกษา: 673380509-6 sec.1

ส่วนที่ 1: Software Design & Principles Explanation (เขียนอธิบาย)

 อธิบายสถาปัตยกรรมและ GRASP Patterns: เขียนอธิบายการแบ่งหน้าที่ของคลาส (Entity, Repository, Service, Controller) ตามหลัก GRASP Patterns (เช่น Controller Pattern, High Cohesion, Low Coupling, Information Expert, Indirection)

การออกแบบระบบแบ่งหน้าที่ของคลาสตามหลัก **GRASP Patterns** เพื่อให้ได้ระเบียบ เรียบร้อย และขยายระบบได้ง่าย ดังนี้

- **Entity (Game):** ทำหน้าที่เป็น **Information Expert** ที่ครอบครองและจัดการข้อมูลของเกม เช่น id, title, price, discountType
- **Repository (GameRepository):** ทำหน้าที่เป็น **Indirection** คอยเป็นตัวกลางเชื่อมระหว่าง Business Logic กับ Database (PostgreSQL) ผ่าน Spring Data JPA ช่วยลดข้อผูกมัด (Low Coupling) กับโครงสร้างตาราง
- **Service (GameService):** มีความเกาะเกี่ยวสูง (**High Cohesion**) ทำหน้าที่จัดการ Business Logic ของระบบเพียงอย่างเดียว เช่น การประสานงานคำนวณราคาสินค้าและการส่งข้อมูลไปบันทึก
- **Controller (GameController):** ทำหน้าที่เป็น **Controller Pattern** ทำการรับคำขอ HTTP Request จาก Browser ตรวจสอบข้อมูลขั้นต้น แล้วส่งต่อการทำงานให้ Service

 อธิบาย High-Level SOLID Principles: เขียนอธิบายการประยุกต์ใช้หลักการ SOLID (SRP, OCP, LSP, ISP, DIP) ในระบบ

- **S - Single Responsibility Principle (SRP):** แยกหน้าที่ชัดเจน คลาส GameController จัดการเกี่ยวกับ HTTP/View, GameService ทำ Business Logic, DiscountContext เลือกกกลยุทธ์ส่วนลด, และ GameRepository จัดการ Database
- **O - Open/Closed Principle (OCP):** รองรับการเพิ่มเงื่อนไขส่วนลดใหม่ๆ (เช่น VIPDiscountStrategy) โดยสร้างคลาสใหม่ที่ชื่อมารับอินเทอร์เฟซ DiscountStrategy ได้ทันที โดยไม่ต้องแก้ไขโค้ดเดิมใน Service
- **L - Liskov Substitution Principle (LSP):** คลาสลูกทุกตัว (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถนำไปใช้งานแทนอินเทอร์เฟซ DiscountStrategy ได้สมบูรณ์โดยไม่ทำให้ระบบทำงานผิดพลาด

- **I - Interface Segregation Principle (ISP):** อินเทอร์เฟซ DiscountStrategy กำหนดเฉพาะเมธอดที่จำเป็นต่อการคำนวณราคาเท่านั้น ไม่ยัดเยียดเมธอดที่ไม่เกี่ยวข้อง
- **D - Dependency Inversion Principle (DIP):** GameService และ GameController ไม่ได้ยึดติดกับ Class ระดับล่างโดยตรง แต่พึ่งพา Abstraction ผ่านการทำ **Constructor Injection**

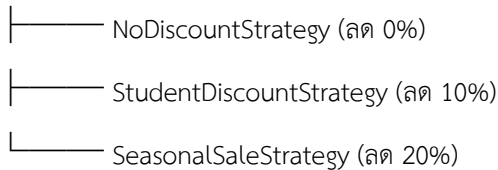


อธิบาย Strategy Pattern: เขียนอธิบายการประยุกต์ใช้ Strategy Pattern ในการคำนวณส่วนลดราคาเกม

(DiscountStrategy, NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy, DiscountContext) พร้อม
 ประโยชน์ด้าน Open/Closed Principle (OCP)

ระบบประยุกต์ใช้ **Strategy Pattern** เพื่อแยกอัลกอริทึมคำนวณส่วนลดออกจาก Business Logic หลัก:

[DiscountStrategy (Interface)]



- **โครงสร้าง:** มี DiscountStrategy เป็น Interface หลัก และมี Concrete Strategies แยกคำนวณตามประเภทส่วนลด
- **การทำงาน:** DiscountContext ทำหน้าที่สวิตช์เลือก Strategy ที่เหมาะสมตามค่า discountType ที่ส่งมาจากฟอร์ม
- **ประโยชน์ต่อ OCP:** หากอนาคตต้องการเพิ่มส่วนลดแบบใหม่ ไม่จำเป็นต้องเขียน if-else ซ้อนกันหลายชั้นใน Service เพียงแค่สร้าง Strategy Class ใหม่ขึ้นมา



อธิบาย Layered Architecture: เขียนอธิบายว่าทำไมต้องแยก Service Layer ออกจาก Controller และ Repository
 ประโยชน์ด้าน Low Coupling และ High Cohesion

การแยก **Service Layer** ออกจาก Controller และ Repository ช่วยสร้างสถาปัตยกรรมที่ดี:

- **Low Coupling (การลดความยึดติด):** GameController ไม่จำเป็นต้องรู้ว่าข้อมูลจัดเก็บใน PostgreSQL อย่างไร หรือใช้สูตรคณิตศาสตร์ใดคำนวณส่วนลด รับหน้าที่เพียงแครับ-ส่งค่าไปหา View
- **High Cohesion (ความเกาะเกี่ยวของหน้าที่):** ทำให้แต่ละ Layer โฟกัสเฉพาะงานของตัวเอง Controller จัดการ UI Flow, Service จัดการ Business Rules, และ Repository จัดการ Data Persistence



อธิบาย Execution Flow: เขียนอธิบายลำดับการทำงาน (Flow) เมื่อมี HTTP Request เข้ามาจาก Browser จนไปถึงการบันทึก/ดึงข้อมูลจาก PostgreSQL และคำนวณส่วนลดผ่าน Strategy Pattern

1. **User Request:** ผู้ใช้กดบันทึกเกมจากหน้าเว็บ ฟอรัมจะส่ง HTTP POST /games/save พร้อมข้อมูล Game มายัง GameController
2. **Controller Routing:** GameController รับ Request และส่ง Object ไปให้ GameService ผ่าน Constructor Injection
3. **Strategy Calculation:** GameService เรียกใช้ DiscountContext เพื่อประมวลผลคำนวณราคาสุทธิหลังหักส่วนลดตาม Strategy ที่เลือก
4. **Database Persistence:** GameService เรียก GameRepository.save() เพื่อบันทึกข้อมูลลงตาราง games ใน PostgreSQL Database (lab7demo)
5. **Response Redirect:** GameController ส่ง redirect:/games เพื่อดึงรายการล่าสุดผ่าน GameRepository.findAll() นำไปแสดงผลบน list.html

ส่วนที่ 2: Code Implementation & Explanation

โครงสร้าง Code พร้อมคำอธิบาย (Entity, Repository, Strategy Package, Service, Controller) โดยอธิบายการใช้ Dependency Injection (Constructor Injection) ในทุก Layer

โครงสร้างโปรเจกต์ (Project Structure):

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── model/
│   └── Game.java
├── repository/
│   └── GameRepository.java
├── strategy/
│   ├── DiscountStrategy.java
│   ├── NoDiscountStrategy.java
│   ├── StudentDiscountStrategy.java
│   ├── SeasonalSaleStrategy.java
│   └── DiscountContext.java
├── service/
│   └── GameService.java
└── controller/
    └── GameController.java
```

อธิบาย:

Model Layer (Game.java)

- **หน้าที่:** เป็น JPA Entity ทำหน้าที่ Map คุณลักษณะของ Object เข้ากับตาราง games ในฐานข้อมูล PostgreSQL
- **การใช้ DI:** ไม่มี เนื่องจากเป็น Data Class/POJO ที่มีไว้เพื่อเก็บข้อมูลระหว่าง Layer เท่านั้น

Repository Layer (GameRepository.java)

- **หน้าที่:** Data Access Layer ที่ extends JpaRepository<Game, Long> เพื่อใช้คำสั่งจัดการฐานข้อมูลสำเร็จรูป (CRUD)
- **การใช้ DI:** Spring Data JPA จะสร้าง Bean ของ Interface นี้ขึ้นมาใน Spring Container ให้อัตโนมัติ เพื่อเตรียมฉีดไปยัง Service Layer

Strategy Package (strategy/*)

- **หน้าที่:** แยกสูตรคำนวณส่วนลดออกจากกันตามหลัก Open/Closed Principle โดยมี DiscountContext ทำหน้าที่เลือกสูตรที่เหมาะสม

- การใช้ DI: คลาส DiscountContext ใส่คำสั่ง @Component ไว้ เพื่อให้ Spring มองเห็นคลาสนี้เป็น Bean และสามารถนำไปส่งใช้งานใน Service ได้

Service Layer (GameService.java)

- **หน้าที่:** จัดการ Business Logic ทั้งหมดของระบบ เช่น ดึงข้อมูล นำราคาไปคำนวณผ่าน Strategy และส่งบันทึก
- **การใช้ DI (Constructor Injection):**

```
private final GameRepository gameRepository;

private final DiscountContext discountContext;

@Autowired

public GameService(GameRepository gameRepository, DiscountContext discountContext) {

    this.gameRepository = gameRepository;

    this.discountContext = discountContext;

}
```

ใช้ Constructor Injection ในการรับ GameRepository และ DiscountContext เข้ามาทำงาน ช่วยลด Loose Coupling ทำให้ไม่ต้องสร้าง instance เองด้วยคำสั่ง new

Controller Layer (GameController.java)

- **หน้าที่:** Presentation Layer ทำหน้าที่รับ HTTP Request จากเบราว์เซอร์ ประมวลผลร่วมกับ Service และส่งค่าไปยัง Thymeleaf
- **การใช้ DI (Constructor Injection):**

```
private final GameService gameService;

public GameController(GameService gameService) {

    this.gameService = gameService;

}
```

ส่ง GameService เข้ามาผ่าน Constructor เพื่อเรียกใช้เมธอดจัดการข้อมูล โดย Controller ไม่จำเป็นต้องรู้รายละเอียดภายในว่า Service ดึงข้อมูลมาจากไหนหรือคำนวณราคายังไง

ส่วนที่ 3: Web Application & Database Screenshots

📌 ข้อกำหนดสำคัญ: ในขั้นตอนการเพิ่มเกมใหม่ นักศึกษาต้องใส่รหัสนักศึกษาและ Section ของตนเอง ลงในข้อมูลเกม (เช่น ในช่องชื่อเกม Title หรือแนวเกม Genre เช่น Elden Ring (663380123-4 Sec 1))

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)

← กลับไปรายการเกม

+ เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

🔗 ชื่อเกม

kitiyada 673380509-6 Section 1

🎮 แนวเกม (Genre) 🖥️ แพลตฟอร์ม (Platform)

Action PS5

☆ คะแนน (Rating) 💰 ราคาปกติ (บาท) % ส่วนลด (Strategy)

9.1 1900 ส่วนลดนักศึกษา (10%)

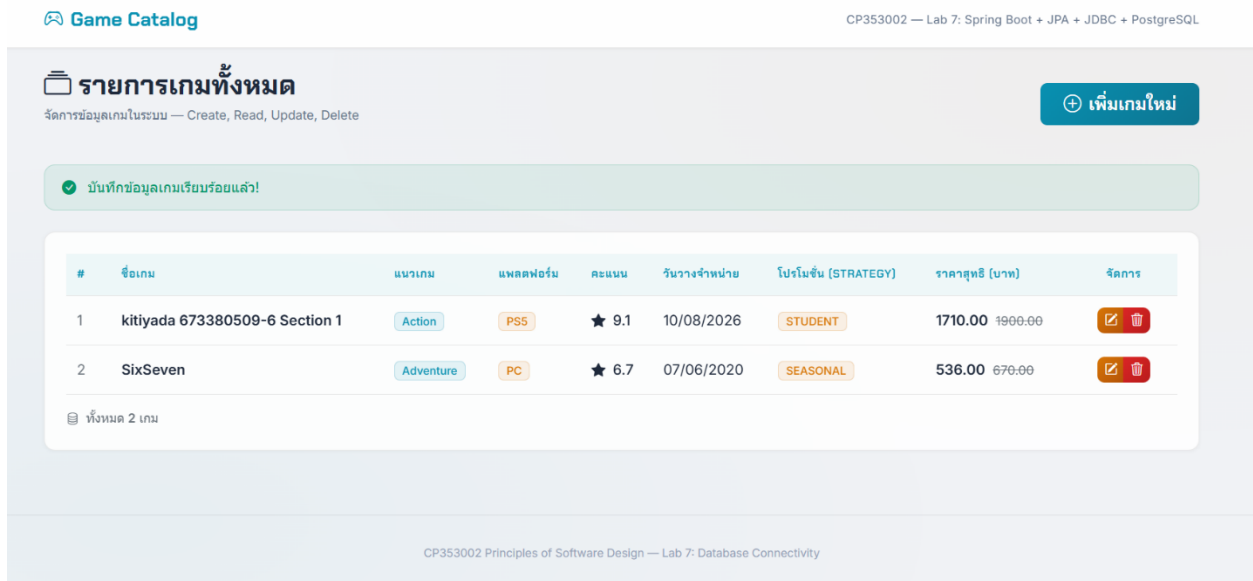
📅 วันวางจำหน่าย (Release Date)

08/10/2026

💾 บันทึก ✕ ยกเลิก

รูปที่ 3.1: หน้าฟอร์มเพิ่มเกมใหม่ (ระบุรหัสนักศึกษา 663380123-4 Section 1 และเลือกส่วนลดนักศึกษา 10%และ20%)

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)



รูปที่ 3.2: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม (Update)

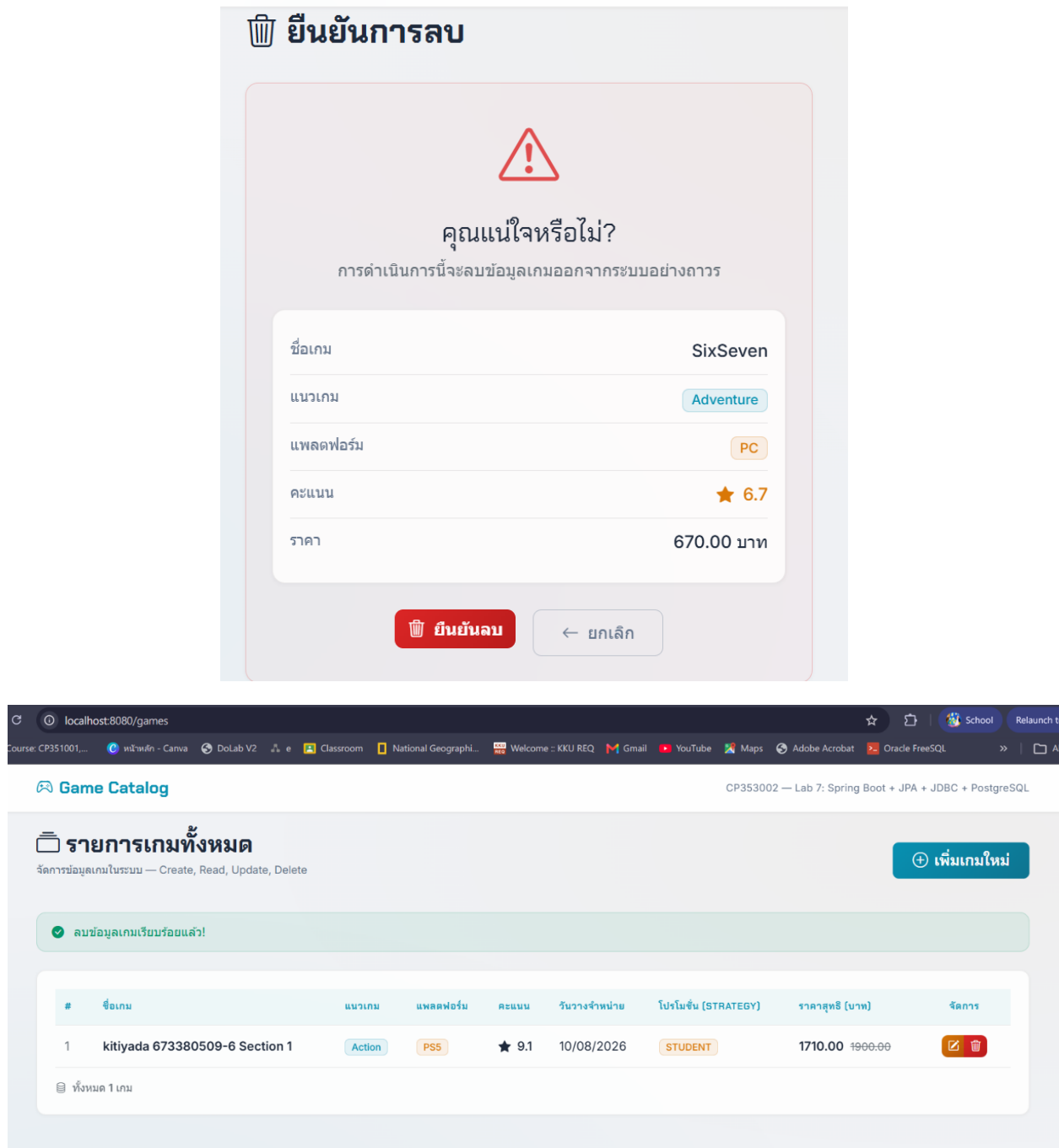
The screenshot shows the 'แก้ไขข้อมูลเกม' (Edit Game Information) form. It includes a back button '← กลับไปรายการเกม' and a title 'แก้ไขข้อมูลเกม: kitiyada 673380509-6 Section 1'. The form fields are:

- ชื่อเกม (Game Name): kitiyada 673380509-6 Section 1
- แนวเกม (Genre): Action
- แพลตฟอร์ม (Platform): PS5
- คะแนน (Rating): 9.1
- ราคาปกติ (บาท) (Regular Price (Baht)): 1900.0
- % ส่วนลด (Strategy) (% Discount (Strategy)): ส่วนลดนักศึกษา (10%)
- วันวางจำหน่าย (Release Date): mm/dd/yyyy

At the bottom, there are two buttons: 'อัปเดต' (Update) and 'ยกเลิก' (Cancel).

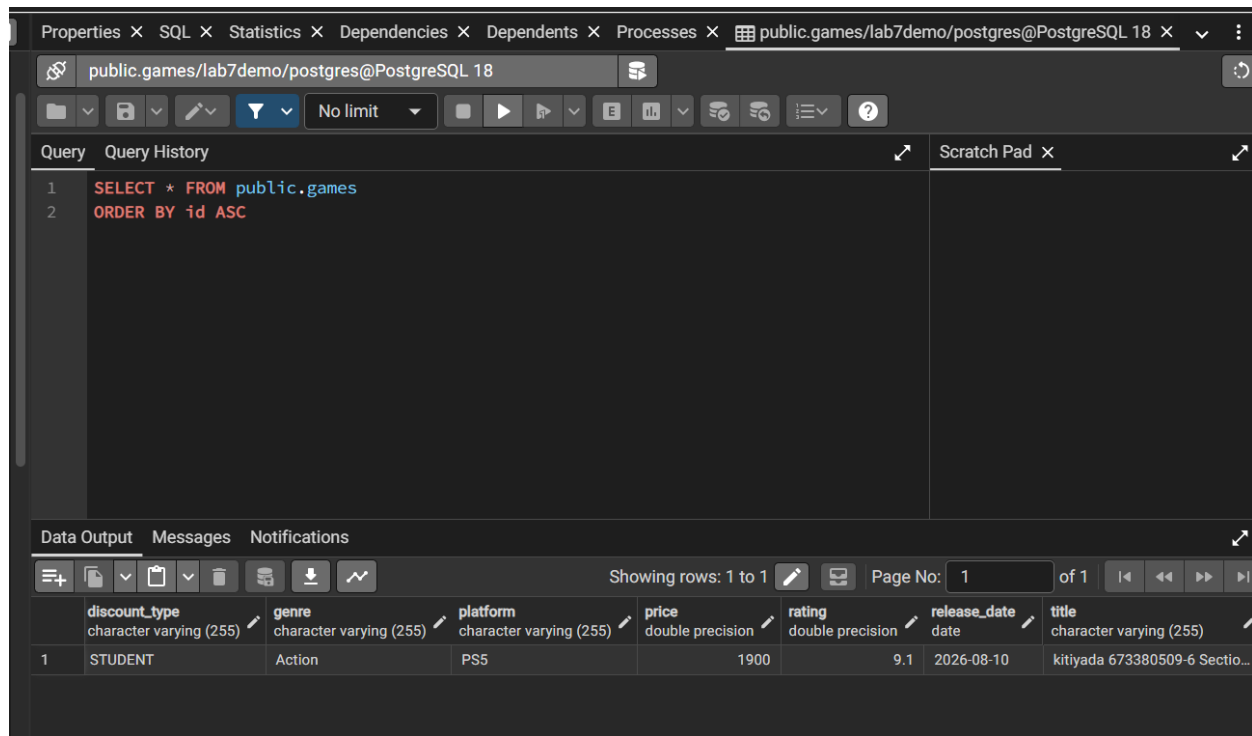
รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลเกม

3.4 หน้าจอยืนยันและการลบข้อมูลเกม (Delete)



รูปที่ 3.4: หน้าจอยืนยันการลบเกม และผลลัพธ์หลังทำการลบ

3.5 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database



The screenshot shows a PostgreSQL database management tool interface. The top bar includes tabs for Properties, SQL, Statistics, Dependencies, Dependents, and Processes. The main window displays a query in the SQL editor:

```
1 SELECT * FROM public.games
2 ORDER BY id ASC
```

Below the query editor, the Data Output tab is active, showing a table with the following data:

	discount_type character varying (255)	genre character varying (255)	platform character varying (255)	price double precision	rating double precision	release_date date	title character varying (255)
1	STUDENT	Action	PS5	1900	9.1	2026-08-10	kityada 673380509-6 Sectio...

รูปที่ 3.5: ข้อมูลตาราง games ที่ถูกจัดเก็บจริงในฐานข้อมูล PostgreSQL